Function 14b: HTML Elements Inventory

This document lists all HTML elements that Function 14b (Upload Thumbnails via Selenium) searches for during automated thumbnail uploads to Alma, in the order they are expected.

Initial Page Load

1. Body Tag

- **Selector**: `By.TAG_NAME, "body"`
- **When**: Immediately after navigation to Alma SSO login page
- **Purpose**: Verify the page has loaded and rendered basic HTML structure
- **Wait Time**: 10 seconds (+ optional 30 second extension if not found)
- **Why**: Ensures Firefox has successfully loaded the page before attempting any interactions. If this fails, user may still be completing SSO/DUO authentication.

Search Configuration (CURRENTLY SKIPPED)

The following elements are **commented out** in the current code because the Alma search bar retains previous settings:

2a. Search Type Dropdown (NOT USED)

- **Selector**: `By.ID, "searchType"`
- **Purpose**: Would set search type to "Digital titles"
- **Status**: ⚠️ Commented out - assumes search bar retains "Digital titles" setting from previous session
- **Why Skipped**: The search dropdown remembers the last selection, so we can skip this step

2b. Search Field Dropdown (NOT USED)

- **Selector**: `By.ID, "searchField"`
- **Purpose**: Would set search field to "Representation PID" or "Representation ID"
- **Status**: ⚠️ Commented out - assumes search bar retains field setting from previous session
- **Why Skipped**: The search dropdown remembers the last selection

⚠️ **IMPORTANT**: User must manually configure these settings BEFORE automation starts:

1. Click search dropdown (left side) → Select "Digital titles"
2. Click field dropdown (middle) → Select "Representation ID" or "Representation PID"
3. Leave search box empty

Search Execution

3. Search Input Field

- **Selector**: `By.ID, "NEW_ALMA_MENU_TOP_NAV_Search_Text"`
- **When**: For each record in the CSV
- **Purpose**: Enter the representation ID to search for
- **Wait Time**: 10 seconds

- **Interactions**:
  1. Check if field is disabled → Enable via JavaScript if needed
  2. Clear existing value via JavaScript: `arguments[0].value = '';`
  3. Set new value via JavaScript: `arguments[0].value = arguments[1];` (rep_id)
  4. Trigger input event: `dispatchEvent(new Event('input', { bubbles: true }))`
  5. Simulate ENTER key: `KeyboardEvent('keydown', {key: 'Enter', ...})`
- **Why JavaScript**: Alma's Angular-based UI can mark fields as "disabled" or fail to detect native Selenium `.send_keys()`. JavaScript injection bypasses these framework restrictions and ensures the value is set and detected.

Note: Before searching, the function runs popup dismissal scripts to clear any "Manage Widgets" or overlay popups that might block the search field.

Search Results Navigation

4. Digital Representations Link

- **Multiple Selectors** (tried in order):

  1. **Primary**: XPath case-insensitive text search
     `By.XPATH, "//*[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'digital representation')]"`
  2. **Fallback 1**: Partial link text
     `By.PARTIAL_LINK_TEXT, "Digital Representation"`
  3. **Fallback 2**: Ex-link with span class
     `By.XPATH, "//ex-link[.//span[contains(@class, 'sel-smart-link-nggeneralsectiontitleall_titles_details_digital_representations')]]"`
  4. **Last Resort**: Any element with "Digital" in text
     `By.XPATH, "//*[contains(text(), 'Digital')]"`

- **When**: After search results page loads (10 second wait)

- **Wait Time**: 15 seconds (primary), then 5 seconds for each fallback

- **Purpose**: Click the "Digital Representations (X)" link to open the modal with representation details

- **Interactions**:

  1. Scroll element into view: `scrollIntoView(true)`
  2. Try regular click
  3. If blocked, use JavaScript click: `arguments[0].click()`

- **Why Multiple Selectors**: Alma's UI rendering can vary based on record type, load timing, or UI updates. Multiple strategies ensure we find the link regardless of how it's rendered.

---

5. Overlay Close Button (Optional)

- **Selector**: `By.CSS_SELECTOR, ".sel-id-ex-svg-icon-close"`
- **When**: After clicking Digital Representations link
- **Purpose**: Close any overlay/popup that might obscure the representation list
- **Wait Time**: No wait (immediate attempt)
- **Status**: Optional - if not found, automation continues normally

- **Why**: Sometimes Alma shows additional overlays or dialog boxes that need to be dismissed before the representation list is fully accessible. This is a defensive check.

Representation Selection

6. Representation ID Link

- **Multiple Selectors** (tried in order):
  1. **Strategy 1**: Exact link text
     `By.LINK_TEXT, rep_id`
  2. **Strategy 2**: Partial link text
     `By.PARTIAL_LINK_TEXT, rep_id`
  3. **Strategy 3**: XPath text search
     `By.XPATH, f"//*[contains(text(), '{rep_id}')]"`
  4. **Strategy 4**: XPath anchor search
     `By.XPATH, f"//a[contains(., '{rep_id}')]"`
- **When**: After Digital Representations modal/page opens
- **Wait Time**: 2 seconds per strategy
- **Purpose**: Click on the specific representation ID to open its detail page
- **Example**: If rep_id is "13312244810006091-1956", find and click the link with that exact text
- **Why Multiple Strategies**: The representation ID might appear as:
  - Plain link text
  - Link with additional formatting/whitespace
  - Text within a table cell
  - Link within a complex nested structure

  Multiple strategies ensure we find it regardless of presentation.

File Upload

7. Thumbnail File Input

- **Selectors**:
  1. **Primary**: `By.ID, "pageBeansavedFile"`
  2. **Fallback**: `By.CSS_SELECTOR, "input[type='file']"`
- **When**: After representation detail page loads
- **Wait Time**: 10 seconds (primary), then 5 seconds (fallback)
- **Purpose**: Upload the processed thumbnail file
- **Interaction**: `send_keys(str(file_path.absolute()))`
- **Why ID-based**: The file input has a specific ID in Alma's UI. Fallback to generic `input[type='file']` ensures compatibility if Alma changes the ID.

- **File Validation**: Before upload, the function verifies:
  - File exists on disk
  - File size > 0 bytes (zero-byte files trigger warning and skip)

Save Changes

8. Save Button

- **Selectors**:
  1. **Primary**: `By.ID, "PAGE_BUTTONS_cbuttonsave"`
  2. **Fallback**: `By.XPATH, "//button[contains(text(), 'Save')]"`
- **When**: After file is selected (2 second wait for processing)
- **Wait Time**: 10 seconds (primary), then 5 seconds (fallback)
- **Purpose**: Submit the form and save the uploaded thumbnail to Alma
- **Interaction**: `click()`
- **Post-Save Wait**: 3 seconds for save operation to complete
- **Why**: The Save button commits the thumbnail upload. After successful save, the record is marked as successful and added to the `successful_mms_ids` set for CSV filtering.

Popup Dismissal Elements (Defensive)

Throughout the automation, the function runs JavaScript scripts to dismiss various popup elements that might interfere with interaction:

9. General Popup Close Buttons

- **Selectors**:
  - `[class*="close"]`
  - `[class*="dismiss"]`
  - `[aria-label*="close" i]`
- **Purpose**: Close generic modal dialogs, overlays, or notification banners
- **Method**: JavaScript click on first visible close button

10. "Manage Widgets" Popup

- **Purpose**: Dismiss Alma's "Manage Widgets" popup that can block the search field
- **Method**: Complex JavaScript that:
  1. Finds buttons/links with "manage" or "widget" in text/aria-label
  2. Searches for parent dialog/modal container
  3. Clicks close button within that container
  4. Also searches for direct widget modal close buttons: `[class*="widget" i] [class*="close"]`

11. Cookie Banners

- **Selectors**: `[class*="cookie"] button`, `[id*="cookie"] button`
- **Purpose**: Accept/dismiss cookie consent banners

- **Method**: JavaScript click on buttons with "accept", "ok", or "agree" text

12. "Stay Signed In" Prompts

- **Purpose**: Dismiss SSO/authentication "Keep me signed in" prompts
- **Method**: JavaScript click on buttons with "no", "not now", "dismiss", or "cancel" text

13. Escape Key Dismissal

- **Purpose**: Close any focused dialog by simulating ESC key press
- **Method**: JavaScript keyboard event dispatch: `KeyboardEvent('keydown', {key: 'Escape', keyCode: 27})`

Debug/Diagnostic Elements (COMMENTED OUT)

The following are **disabled** in the current code to prevent filling up the Downloads folder, but can be re-enabled for debugging:

Screenshots

- **When**: After various page loads (search results, modal opens, errors)
- **File Pattern**: `~/Downloads/alma_*_{timestamp}.png`
- **Purpose**: Visual inspection of page state

HTML Source Dumps

- **When**: After various page loads or errors
- **File Pattern**: `~/Downloads/alma_*_{timestamp}.html`
- **Purpose**: Inspect actual HTML structure to update selectors

---

Summary: Element Flow

For each record in the CSV:

1. ✓ **Body** - Verify page is loaded
2. ⊗ **Search Type Dropdown** - (Skipped - retained from previous session)
3. ⊗ **Search Field Dropdown** - (Skipped - retained from previous session)
4. 🔍 **Search Input** - Enter representation ID
5. 🔗 **Digital Representations Link** - Open representation list
6. ❌ **Overlay Close Button** - (Optional) Dismiss any overlays
7. 🎯 **Representation ID Link** - Select specific representation
8. 📁 **File Input** - Upload thumbnail file
9. 💾 **Save Button** - Commit changes
10. ✓ **Success** - Record added to successful_mms_ids set

Defensive Actions Throughout:

- Popup dismissal scripts run before search (step 4)
- Multiple selector strategies for reliability (steps 5, 7, 8, 9)
- JavaScript-based interaction where native Selenium fails (step 4)
- Stop-on-first-failure for debugging (any exception breaks loop)

Timeout Strategy

- **General Wait**: 2-15 seconds depending on element criticality
- **Search Results Load**: 10 seconds (hardcoded `time.sleep(10)`)
- **Modal/Dialog Wait**: 2 seconds after interactions
- **Save Processing**: 3 seconds after clicking Save

Why Variable Timeouts: Critical elements (body, login detection) get longer waits. Secondary elements (fallback selectors) get shorter waits since we try multiple strategies.

Failure Modes

The automation stops immediately on first failure when encountering:

1. **File not found** - CSV references non-existent file
2. **Zero-byte file** - File is empty (⚠️ WARNING + skip, not stop)
3. **Timeout** - Element not found within wait time
4. **NoSuchElement** - Element doesn't exist on page
5. **Generic Exception** - Unexpected error (logged with full traceback)

On failure, the function:

- Logs detailed error message
- Reports success count, failure count, and remaining count
- Breaks the processing loop
- Leaves Firefox open for manual inspection
- Creates a "failed" CSV if there were some successes

CSS Selectors Reference

Quick reference of all CSS-based selectors used:

Element	Selector	Type
Search Input	`#NEW_ALMA_MENU_TOP_NAV_Search_Text`	ID
Overlay Close	`.sel-id-ex-svg-icon-close`	Class
File Input (primary)	`#pageBeansavedFile`	ID
File Input (fallback)	`input[type='file']`	Type attribute
Save Button (primary)	`#PAGE_BUTTONS_cbuttonsave`	ID
General Close Buttons	`[class*="close"], [class*="dismiss"], [aria-label*="close" i]`	Attribute substring
Cookie Buttons	`[class*="cookie"] button, [id*="cookie"] button`	Attribute substring + tag

XPath Selectors Reference

Element	XPath	Purpose
Digital Reps (primary)	`//*[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'digital representation')]`	Case-insensitive text search
Digital Reps (fallback 2)	`//ex-link[.//span[contains(@class, 'sel-smart-link-nggeneralsectiontitleall_titles_details_digital_representations')]]`	Alma-specific component
Digital Reps (last resort)	`//*[contains(text(), 'Digital')]`	Any element with "Digital"
Rep ID (strategy 3)	`//*[contains(text(), '{rep_id}')]`	Any element containing rep ID
Rep ID (strategy 4)	`//a[contains(., '{rep_id}')]`	Anchor containing rep ID
Save Button (fallback)	`//button[contains(text(), 'Save')]`	Button with "Save" text

Notes

- **JavaScript Dependency**: Many interactions use JavaScript injection instead of native Selenium methods for reliability with Alma's Angular-based UI
- **AppleScript Integration**: macOS-specific window activation for Firefox focus (3 attempts, 1 second apart)
- **Popup Dismissal**: Runs 3 rounds of 6 different dismissal scripts at startup, plus targeted dismissal before each search
- **Browser Left Open**: Firefox is NOT closed after automation completes, allowing manual review of final state
- **CSV Filtering**: Successful uploads are tracked in `successful_mms_ids` set and filtered out of retry CSV
- **Auto-Population**: Failed CSV path is returned to caller (line 5430) for automatic UI field population

---

Version Information

- **Document Created**: March 3, 2026
- **Code Version**: app.py (6,031 lines)
- **Function Location**: app.py
- **Related Function**: prepare_clientthumb_thumbnails() - Function 14a that generates the input CSV